Matching Cyber Security Ontologies through K-Means Algorithms and Deep Learning

Programa de Pós-Graduação em Informática (PPGI)
Aluno: Roberto Monteiro Dias
Professor: Pedro Nuno de Souza Moura
Disciplina: **Aprendizagem Profunda**

Conteúdo

1 Motivação **1**

2 Dataset **2**

3 Definição da Tarefa **2**

3.1 Algoritmo de Avaliação de Correspondências em Clusters Utilizando Redes Neurais Artificiais 3

3.1.1 Avaliação da correspondência 3

4 Rede Neural artificial Utilizada **3**

4.1 Deep Learning 3

4.1.1 Perceptron Multicamadas (MLP) 4

5 Experimento **5**

5.1 Execução da Tarefa de Classificação Binária por meio de Rede Neural artificial 5

5.2 Impressão dos Clusters de Correspondências 5

5.3 Escolha da Correspondência para Avaliação 5

5.4 Treinamento das Redes Neurais 5

5.5 Avaliação do Especialista 6

5.6 Resultado Final 6

5.7 Diagrama de Atividades para demonstração da solução pro meio da rede neural artifical MLP 7

5.8 Representação de Entrada Adotada 7

5.8.1 Definição das Features 7

5.9 Representação da rede artificial MPL no processo de avaliação da correspondencia 8

5.9.1 Camada Densa (Dense Layer) 8

5.9.2 Camada de Saída (Output Layer) 8

5.10 Treinamento da MLP e Cálculo de Probabilidades 9

5.10.1 O Algoritmo de Gradiente Descendente Estocástico (SGD) . . 9

5.10.2 Função de loss Log-Loss (Entropia Cruzada) 9

5.10.3 Probabilidades no Contexto da Log-Loss 10

5.10.4 Predição para Dados Não Classificados 10

5.10.5 Configuração de Entrada de Dados na Rede Neural MLP . . . 10
5.10.6 Saída da Rede 10

6 Exemplo Prático com a Correspondência Selecionada 11
6.1 Exemplo de execução 12
6.1.1 Resultados da avaliação 12
6.1.2 Detalhamento das Etapas 12

7 Hiperparâmetros do Modelo - Biblioteca Java Deeplearning4j 13

8 Implementação 14

9 Alinhamento gerado pelo Programa *View Interactive Ontology Macthing* - ontologia de origem *cmt* e ontologia de destino *Conference* em RDF/XML 15

10 Resultado da avaliação do alinhamento por meio do alinhamento de referência 18

11 Considerações Finais 19

1. Motivação

Mudanças no cenário econômico, político e social impulsionaram uma intensa complexidade nos sistemas de informação (SI), especialmente em setores críticos, como usinas nucleares, que possuem em suas plantas a integração de Sistemas Ciberfísicos (do inglês, *Cyber-Physical System*, ou CPS) [Ghazani et al. 2012, Zeadally e Jabeur 2016]. Este tipo de sistema complexo está ligado ao conceito de Indústria 4.0 [Zeng et al. 2020], que procura combinar componentes cibernéticos e físicos utilizando tecnologias de sensores, computação e redes [Alguliyev et al. 2018]. Ele é empregado em Internet das Coisas (*IoT*), Big Data, tecnologias inteligentes, computação em nuvem, entre outros.

O CPS é a base para o desenvolvimento de diversas áreas, como manufatura inteligente, medicina inteligente, edifícios e infraestruturas inteligentes, cidades inteligentes, veículos inteligentes, dispositivos vestíveis, sistemas móveis, sistemas de defesa, meteorologia, etc. [Alguliyev et al. 2018].

O rápido crescimento de aplicativos CPS tem gerado uma série de problemas relacionados à segurança e à confidencialidade, especialmente em questões envolvendo a integração de infraestruturas críticas e funcionalidades sensíveis [Alguliyev et al. 2018]. Por exemplo, quando as plantas de usinas nucleares, como Angra 1, Angra 2 e, futuramente, Angra 3, da Eletronuclear[1], precisarem interoperar conjuntamente, a interconectividade e a interação devem ser preservadas para garantir a gestão e o controle dos seus processos e sistemas computacionais [Lin e Haga 2021, Santos e Mello 2022, Xue et al. 2022].

Nesse contexto, a ocorrência de falhas de segurança da informação, como loss ou acesso não autorizado a dados confidenciais, durante a comunicação entre os sistemas ciberfísicos e as respectivas plantas dessas usinas, pode colapsar o sistema como um todo e acarretar diversos problemas para o negócio. Para mitigar tais riscos, é essencial implementar mecanismos e/ou tecnologias que assegurem que a comunicação entre os sistemas e suas respectivas plantas seja tratada sob o prisma da segurança da informação: confiabilidade, integridade e disponibilidade [ISO 2013]. Uma solução promissora é o uso de ontologias do domínio de segurança, que permitem construir um modelo de conhecimento compartilhado entre uma planta de usina nuclear e seus sistemas ciberfísicos.

No entanto, as ontologias do domínio de segurança possuem diferentes requisitos e interesses, o que as torna suscetíveis ao problema da heterogeneidade. Isso se manifesta na incompatibilidade semântica entre duas ontologias de segurança para CPS, gerando resultados imprevisíveis, ambíguos ou até mesmo incompletos, que podem abrir brechas para que um atacante experiente insira códigos maliciosos nesses sistemas [de Sá et al. 2017, Mozzaquatro et al. 2018].

Por essa razão, é necessário realizar um processo de alinhamento de ontologias para solucionar os problemas de heterogeneidade na integração de ontologias do domínio de segurança de plantas de usinas nucleares (i.e., Angra 1 e Angra 2). Esse alinhamento permitirá o pleno funcionamento de operações integradas dessas usinas, sem possíveis ataques cibernéticos.

[1]https://www.eletronuclear.gov.br/Nossas-Atividades/Paginas/default.aspx

Para alcançar esse objetivo, foi desenvolvido um Programa denominado *View Interactive Ontology Matching*, focado no alinhamento interativo de ontologias, utilizando a técnica de clusterização em conjunto com uma rede neural artificial MLP. O Programa emprega o algoritmo K-Means para agrupar as correspondências em clusters, selecionando uma correspondência de cada cluster para ser avaliada pela respectiva rede neural artificial MLP.

2. Dataset

O dataset utilizado foi o da trilha de Conferências Acadêmicas da OAEI (do inglês,*Ontology Alignment Evaluation Initiative* , ou Iniciativa de Avaliação de Alinhamento de Ontologias) [2]. A OAEI disponibiliza alinhamentos de referência para diversos domínios de avaliação. O dataset *Conference* é composto por 16 ontologias, com alinhamentos de referência estabelecidos para cada par de ontologias. Sete dessas ontologias estão envolvidas nos alinhamentos de referência: Cmt, Conference, ConfOf, Edas, Ekaw, Iasted e Sigkdd, totalizando 21 alinhamentos de referência [Zamazal e Svátek 2017]. Para o experimento descrito neste trabalho, serão utilizadas as sete ontologias relacionadas no alinhamento de referência desse respectivo dataset conforme Tabela 1.

Nome	Classes	Propriedades de Dados	Propriedade de Objetos	Link de Acesso
Cmt	36	10	49	http://msrcmt.research.microsoft.com/cmt
Confious	57	5	52	http://www.confious.com
Conference	38	23	13	http://www.conftool.net
Edas	104	20	30	http://edas.info/
Ekaw	74	0	33	http://ekaw.vse.cz
Iasted	140	3	38	http://iasted.com/conferences/2005/cancun/ms.htm
Sigkdd	49	11	17	http://www.acm.org/sigs/sigkdd/kdd2006

Tabela 1. Tabela com as sete ontologias relacionadas no alinhamento de referência do Conference Dataset.

3. Definição da Tarefa

A tarefa descrita consiste em uma abordagem de classificação binária aplicada ao alinhamento de ontologias, utilizando rede neural artificial. O objetivo central é determinar se uma correspondência entre conceitos de duas ontologias é válida (*true*) ou inválida (*false*). O processo inicia-se com o carregamento das ontologias e a geração das correspondências, seguida pelo cálculo das medidas de similaridade entre elas. Em seguida, o algoritmo aplica a técnica de *clusterização* K-Means para organizar as correspondências em grupos com base nas similaridades calculadas.

No decorrer do processo iterativo, as correspondências de cada *cluster* são avaliadas por uma rede neural artificial: a rede Perceptron Multicamadas (MLP). Essa rede analisa a correspondência de maneira independente, gerando um resultado binário (*true* ou *false*), que define a classificação da correspondência.

[2]https://oaei.ontologymatching.org/

3.1. Algoritmo de Avaliação de Correspondências em Clusters Utilizando Redes Neurais Artificiais

3.1.1. Avaliação da correspondência

1. Submeter a correspondência à rede Perceptron Multicamadas (MLP) para avaliação:
    - Se a MLP retornar "false", então a correspondência é marcada como **false.**
    - Se a MLP retornar "true", então a correspondência é marcada como **true.**

4. Rede Neural artificial Utilizada

4.1. Deep Learning

O *Deep Learning* é um modelo computacional inspirado no funcionamento do cérebro humano, que, como parte da Inteligência Artificial (IA), foi desenvolvido a partir de redes neurais artificiais (RNA) [Zengeya e Vincent Fonou-Dombeu 2024]. Assim como no cérebro, essas redes possuem unidades de processamento interconectadas, chamadas neurônios, organizadas em camadas. Essas camadas incluem uma camada de entrada, responsável por receber os dados; camadas ocultas, que processam as informações; e uma camada de saída, que gera os resultados desejados. Nas camadas ocultas, são extraídos padrões e representações complexas dos dados de entrada [Qamar e Zardari 2023]. O *Deep Learning* é capaz de identificar relações sofisticadas entre as informações, permitindo uma compreensão profunda dos dados [Goodfellow et al. 2016].

Atualmente, diversas aplicações de técnicas de *Deep Learning* têm ganhado destaque, como:

- **Automóveis Autônomos**: processamento de dados de sensores e câmeras para navegação e tomada de decisões em tempo real;
- **Jogos**: desenvolvimento de agentes autônomos capazes de aprender a jogar e vencer jogos complexos, como xadrez e Go;
- **Processamento de Linguagem Natural (NLP)**: tradução automática, chatbots, análise de sentimentos e geração de texto;
- **Reconhecimento de Voz**: conversão de fala em texto e sistemas de transcrição, permitindo interações mais naturais com dispositivos;
- **Recomendações**: algoritmos que sugerem produtos ou conteúdos com base no comportamento do usuário; e
- **Saúde**: análise de exames médicos, previsão de doenças e descoberta de medicamentos.

Essas aplicações demonstram o impacto significativo do *Deep Learning* em várias áreas, transformando a forma como os seres humanos interagem com a tecnologia [Qamar e Zardari 2023].

O *Deep Learning* utiliza redes neurais para capacitar máquinas a aprender e tomar decisões inteligentes. Diferente dos algoritmos tradicionais de aprendizado

de máquina, que dependem de Programação explícita para realizar tarefas, os algoritmos de *Deep Learning* aprendem diretamente a partir dos dados [Tyagi 2024]. Eles identificam padrões complexos e representações detalhadas, o que os ajuda a generalizar e a lidar com tarefas desafiadoras.

Exemplos de redes neurais artificiais incluem o *Perceptron Multicamadas* (do inglês, *Multi-Layer Perceptron* ou MLP), as redes *Convolucionais* (do inglês, *Convolutional Neural Networks* ou CNNs) e as redes *Recorrentes* (do inglês, *Recurrent Neural Networks* ou RNNs) [Tyagi 2024].

4.1.1. Perceptron Multicamadas (MLP)

O Perceptron Multicamadas (do inglês, *Multi-Layer Perceptron* ou MLP) não se refere a um simples perceptron com múltiplas camadas. Embora seja tecnicamente composto por vários perceptrons, o perceptron, por definição, é uma rede neural projetada exclusivamente para classificação binária [Goodfellow et al. 2016]. É uma evolução do Perceptron para resolução de problemas em casos em que o conjunto de dados não são linearmente separáveis. Diferentemente do perceptron de camada única, o MLP apresenta a capacidade de aprender padrões e relacionamentos complexos nos dados. Essa habilidade decorre do uso de funções de ativação e da inclusão de várias camadas ocultas, permitindo que o modelo capture a não linearidade presente nas informações [Cunha 2022].

O MLP destaca-se por sua versatilidade, sendo amplamente utilizado em uma variedade de tarefas, principalmente para processos de classificação em *Deep Learning*. Ele desempenha um papel essencial como bloco de construção no aprendizado profundo, devido à sua proficiência em modelar relações complexas nos dados, o que reforça sua relevância no campo da inteligência artificial [Cunha 2022].

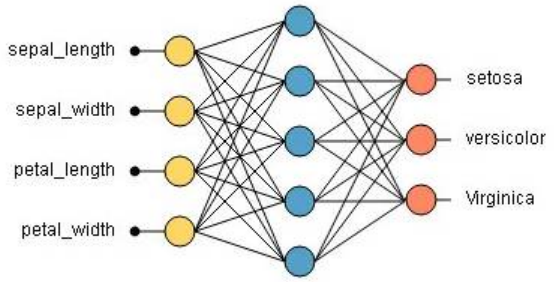

Figura 1. Exemplo de um modelo adaptado para classificar flores Iris usando o MLP com uma camada oculta de 5 neurônios e 3 neurônios na camada de saída, correspondendo às classes Setosa, Virginica e Versicolor[3].

[3]https://medium.com/ensina-ai/rede-neural-perceptron-multicamadas-f9de8471f1a9

5. Experimento

5.1. Execução da Tarefa de Classificação Binária por meio de Rede Neural artificial

O processo de avaliação de correspondências entre ontologias utilizando redes neurais artificiais pode ser detalhado da seguinte maneira:

5.2. Impressão dos Clusters de Correspondências

O Programa inicia exibindo os clusters das correspondências, apresentando o agrupamento das entidades da ontologia de origem e da ontologia de destino. No exemplo, utilizam-se a ontologia de origem (*cmt*) e a ontologia de destino (*conference*). Cada linha representa uma correspondência entre as classes da ontologia de origem e as classes da ontologia de destino, acompanhada das métricas de similaridade associadas a cada par.

Exemplo de um par de correspondências gerado pelo Programa *view*:

46º Interação
Imprime os Clusters das Correspondências:
11,ProgramCommittee#Program_committee,0,0,2.62,0.87,1.86
21,ConferenceChair#Conference,0,0,2.56,0.85,1.66
01,ExternalReviewer#Reviewer,0,0,2.62,0.87,1.66
11,hasBid#has_a_URL,0,0,2.62,0.87,1.66
21,Administrator#Abstract,0,0,2.54,0.85,1.56
01,SubjectArea#Submitted_contribution,0,0,2.59,0.86,2.15
01,hasBeenAssigned#has_been_assigned_a_review_reference,0,0,2.59,0.86,1.72
01,AssociatedChair#Chair,0,0,2.59,0.86,2.15
01,Meta-Reviewer#Reviewer,0,0,2.59,0.86,1.56
01,PaperAbstract#Abstract,0,0,2.59,0.86,1.56
11,reviewerBiddingStartedBy#reviews,0,0,2.62,0.87,2.15
11,assignedByAdministrator#has_been_assigned_a_review_reference,0,0,2.62,0.87,2.15
11,runPaperAssignmentTools#is_a_full_paper_submission_date,0,0,2.62,0.87,2.82
11,date#is_an_ending_date,0,0,2.62,0.87,3.81
11,writtenBy#is_given_by,0,0,2.62,0.87,3.81
11,submitPaper#is_a_full_paper_submission_date,0,0,2.62,0.87,3.81
11,title#has_a_commtitee,0,0,2.62,0.87,2.82
11,logoURL#has_a_URL,0,0,2.62,0.87,2.82

5.3. Escolha da Correspondência para Avaliação

A correspondência selecionada para avaliação é a do **Cluster 21**, especificamente a correspondência `ConferenceChairConference`. Esta correspondência será analisada por uma rede neural, conforme detalhado na tarefa de avaliação especializada.

5.4. Treinamento das Redes Neurais

Durante o processo de avaliação, as redes neurais treinam e a loss (ou erro) do modelo é monitorada. O Programa exibe a evolução da loss durante as iterações (epoch), o que reflete o processo de aprendizado do modelo ao longo do tempo.

Evolução da loss: As métricas de loss para as epoch de treinamento indicam que o modelo está aprendendo a partir dos dados, com a loss diminuindo ao longo das iterações:

- **Epoch 0**: Loss = 0.32009349149816174
- **Epoch 1**: Loss = 0.22066931631050857
- **Epoch 2**: Loss = 0.1579148161645029
- **Epoch 3**: Loss = 0.12332869511024624
- **Epoch 4**: Loss = 0.10277950062471278
- **Epoch 5**: Loss = 0.09037146848790786
- **Epoch 6**: Loss = 0.08134754031312232
- **Epoch 7**: Loss = 0.07432893678253773
- **Epoch 8**: Loss = 0.06901330106398638
- **Epoch 9**: Loss = 0.06471601187014113

A *Loss* é uma medida da diferença entre a previsão do modelo e o valor real (neste caso, a correspondência correta ou incorreta), e quanto menor a *Loss*, melhor o desempenho do modelo.

5.5. Avaliação do Especialista

Após o treinamento, a correspondência é avaliada por um especialista. Este especialista atribui um valor binário (`True` ou `False`) indicando se a correspondência é válida.

Avaliação dos Especialistas:

- **Especialista**: True

5.6. Resultado Final

Com base na avaliação do especialista, que concorda que a correspondência é válida (`True`), o **resultado final** é determinado como `True`. Isso indica que a correspondência entre `ConferenceChair` e `Conference` foi considerada correta pelo especialista, e o modelo de redes neurais aprendeu a identificar corretamente essa correspondência.

```
System.out.println("Escolha a correspondencia para avaliacao do Especialista:");
System.out.println();
// Exibe o cluster e a correspond ncia
System.out.println("Cluster: " + value.getCluster());
System.out.println("Correspond ncia: " + value.getNome());
System.out.println();
// Criação de uma lista de correspond ncias candidatas
List<CorrespondenciaCandidata> correspondenciasCandidatas = new ArrayList<>();
correspondenciasCandidatas.add(value);
System.out.println("Avaliação do Especialista: ");
System.out.println();
// Criação do classificador
DenseMLPClassifier r = new DenseMLPClassifier();
```

```
   \multicolumn{4}{c}{ } \\
// Classificação das entidades
String tipo1 = r.classificarParesEntidades(
   correspondenciasCandidatas, listClassificada);
System.out.println();
// Exibe o resultado da avaliação do especialista
System.out.println("Especialista: " + tipo1);
System.out.println();
// Armazenamento do resultado em uma variável
String tipoAvaliado = tipo1;
```

Código 1. Trecho do código de avaliação da correspondência selecionada.

5.7. Diagrama de Atividades para demonstração da solução pro meio da rede neural artifical MLP

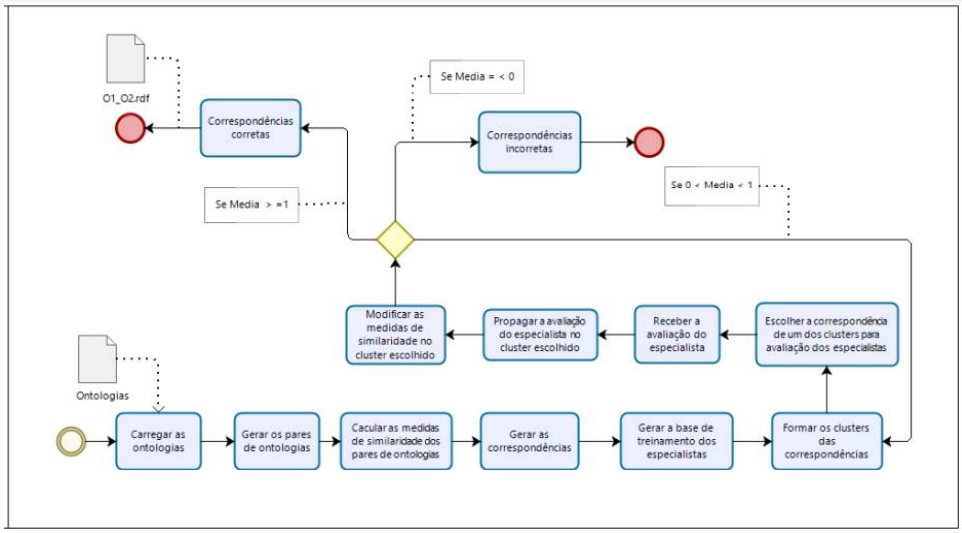

Figura 2. Diagrama de atividades - Programa de alinhamento interativo de ontologias - *View Interactive Ontology Matching*.

5.8. Representação de Entrada Adotada

Trata-se de uma tarefa de classificação binária.

5.8.1. Definição das Features

O modelo utiliza um conjunto de *features* ou características para treinar a rede neural. A seguir, são apresentadas as *features* da correspondência selecionada:

As *features* de entrada (input features) são definidas com base nas correspondências entre ontologias de origem e destino. Para cada instância de treinamento (objeto do tipo 'CorrespondenciaSelecionada'), temos 6 **features** que representam as características das ontologias. Elas são:

- **Cosine**: Similaridade de cosseno entre as ontologias.
- **NGram**: Distância N-Gram, que é uma forma de calcular a sobreposição entre sequências de n caracteres.
- **Lin**: Similaridade semântica calculada via WordNet.
- **Max**: O valor máximo das três similaridades (**Cosine**, **NGram**, **Lin**).
- **MedMax**: A média das três similaridades (**Cosine**, **NGram**, **Lin**).
- **Dem**: A distância Euclidiana entre as ontologias.

5.9. Representação da rede artificial MPL no processo de avaliação da correspondencia

A rede neural utilizada é uma rede artificial MPL, composta por duas camadas principais [Krohn et al. 2019] [Aggarwal et al. 2018]:

- **Camada Densa (Dense Layer)** com ativação **ReLU.**
- **Camada de Saída (Output Layer)** com ativação **Softmax.**

5.9.1. Camada Densa (Dense Layer)

A fórmula geral para a camada densa é dada por:

$$z = W \cdot x + b$$

Onde:

- z é a saída da camada (vetor de somatórios ponderados).
- W é a matriz de pesos.
- x é o vetor de entradas (features).
- b é o vetor de bias.

A ativação **ReLU** é aplicada sobre a saída z da camada densa, dada por:

$$a = \max(0, z)$$

Ou seja, para cada elemento de z, se for negativo, ele é substituído por zero. Se for positivo, ele é mantido.

5.9.2. Camada de Saída (Output Layer)

A camada de saída aplica a função **Softmax**, que transforma a saída da rede em probabilidades. A fórmula do **Softmax** para cada classe c é dada por:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Onde:

- z_i é o valor da saída da rede para a classe i.

- e é a base do logaritmo natural.
- A soma no denominador é sobre todas as saídas da rede para todas as classes possíveis.

5.10. Treinamento da MLP e Cálculo de Probabilidades

O treinamento de uma rede neural, como uma **Multi-Layer Perceptron (MLP)**, envolve o ajuste de **pesos** e **bias** para que o modelo aprenda a realizar previsões precisas com base nos dados de entrada. O processo de treinamento é feito utilizando o **algoritmo de gradiente descendente estocástico** (SGD), que ajusta iterativamente esses parâmetros para minimizar uma função de erro, também chamada de **função de loss** [Krohn et al. 2019] [Aggarwal et al. 2018].

5.10.1. O Algoritmo de Gradiente Descendente Estocástico (SGD)

O **gradiente descendente estocástico** (SGD) é um método de otimização que visa encontrar os parâmetros (pesos e bias) da rede neural que minimizam a função de erro (ou "loss"). A cada iteração do SGD, o algoritmo faz uma atualização nos pesos da rede com base no gradiente da função de erro em relação aos pesos, calculado para um único exemplo ou um pequeno lote de exemplos de treinamento.

A atualização dos pesos é dada pela seguinte fórmula:

$$w = w - \eta \cdot \nabla_w \text{Loss}$$

Onde:

- w é o vetor de pesos da rede.
- η é a taxa de aprendizado, que controla o tamanho dos passos.
- $\nabla_w \text{Loss}$ é o gradiente da função de erro em relação aos pesos.

5.10.2. Função de loss Log-Loss (Entropia Cruzada)

Em problemas de **classificação binária**, a função de erro mais comum é a **log-loss** (também conhecida como **entropia cruzada**). A log-loss mede a discrepância entre as probabilidades previstas pela rede neural ($\hat{y}$) e as probabilidades reais (y).

A fórmula para a **log-loss** de um único exemplo é:

$$\text{Loss} = -\left(y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})\right)$$

Onde:

- y é o rótulo verdadeiro (valor esperado), que pode ser 0 ou 1.
- $\hat{y}$ é a **probabilidade** prevista pela rede neural de que o exemplo pertença à classe 1.

O objetivo do treinamento é **minimizar a log-loss** (ou entropia cruzada) ao longo de todos os exemplos de treinamento. Isso significa ajustar os pesos da rede de forma que a probabilidade $\hat{y}$ prevista pela rede seja o mais próxima possível do valor real y.

5.10.3. Probabilidades no Contexto da Log-Loss

Durante o treinamento, a rede neural gera uma **probabilidade prevista** ($\hat{y}$) para cada exemplo. Essa probabilidade está entre 0 e 1 e reflete a confiança do modelo de que o exemplo pertence à classe 1.

- Quando a rede prevê uma alta probabilidade ($\hat{y} \approx 1$) para um exemplo de classe 1, a loss será baixa se o rótulo verdadeiro y for também 1.
- Se a rede prever uma baixa probabilidade ($\hat{y} \approx 0$) para um exemplo de classe 1, mas o rótulo verdadeiro y for 1, a loss será alta.

Esse cálculo é feito para todos os exemplos no conjunto de treinamento e a soma das losss é minimizada usando o gradiente descendente estocástico.

5.10.4. Predição para Dados Não Classificados

Após o treinamento da rede neural, ela pode ser utilizada para fazer predições em **novos dados não classificados**. Para esses novos exemplos, a rede calculará a probabilidade $\hat{y}$ de que cada exemplo pertença à classe 1. O modelo então atribuirá a classe 1 ao exemplo se $\hat{y}$ for maior que um determinado threshold (geralmente 0.5).

Por exemplo, se a rede prever $\hat{y} = 0.8$, isso significa que a rede está 80% confiante de que o exemplo pertence à classe 1. Se a previsão de probabilidade $\hat{y}$ for inferior a 0.5, o exemplo será classificado como pertencente à classe 0.

Essa abordagem de **probabilidade** é uma característica chave dos modelos de classificação baseados em redes neurais, como a MLP, pois ela fornece não apenas a classe prevista, mas também a confiança ou incerteza associada a essa previsão.

5.10.5. Configuração de Entrada de Dados na Rede Neural MLP

A entrada de dados para a rede neural é construída da mesma forma que as instâncias de treinamento. As **features** são coletadas para a entrada da rede e transformadas em um vetor x de tamanho 6, contendo as características relevantes (**Cosine, NGram**, **Lin, Max, MedMax** e **Dem**). A fórmula para a entrada x é:

$$x = [\text{Cosine}, \text{NGram}, \text{Lin}, \text{Max}, \text{MedMax}, \text{Dem}]$$

5.10.6. Saída da Rede

A rede gera uma saída $\hat{y}$, que representa a probabilidade da classe "True". Para uma classificação binária, o valor de $\hat{y}$ será entre 0 e 1, e o threshold de decisão (threshold) é utilizado para determinar a classe final.

Por exemplo, se $\hat{y}$ for maior que o threshold de decisão (0.3, como definido no código), o par será classificado como "True". Caso contrário, será classificado como "False":

$$\hat{y} \geq \text{threshold} \Rightarrow \text{classificação} = \text{"True}$$

Caso contrário:

$$\hat{y} < \text{threshold} \Rightarrow \text{classificação} = \text{"False}$$

6. Exemplo Prático com a Correspondência Selecionada

Vamos aplicar o processo descrito para uma instância de teste, onde o objetivo é avaliar a correspondência entre pares de ontologias. Suponha que temos a seguinte instância de teste com os seguintes dados:

- **M1**: 0.8
- **M2**: 0.7
- **M3**: 0.9
- **MedMax**: 0.85
- **Dem**: 0.5
- **Alinha**: "True"

Esses dados representam um conjunto de **features** (características) que descrevem a similaridade e a distância entre duas ontologias. O vetor de **features** x para essa instância será:

$$x = [0.8, 0.7, 0.9, 0.85, 0.5]$$

A rede neural processa esse vetor de entrada, e após passar pela camada densa com a função de ativação **ReLU** e pela camada de saída com **Softmax**, gera uma probabilidade $\hat{y}$ de que a correspondência seja classificada como *"True"* (verdadeira).

Etapas de avaliação da correspondência

1. Entrada dos dados: O vetor de **features** x é passado pela rede neural. O MLP (Multi-Layer Perceptron) contém camadas densas que processam as informações.

2. Camadas da Rede: - Camada densa com ReLU: A primeira camada realiza uma transformação linear das entradas, e a função **ReLU** (Rectified Linear Unit) é aplicada para introduzir não-linearidade no modelo. A fórmula da ativação para um neurônio i é dada por:

$$\text{ReLU}(z) = \max(0, z)$$

Onde z é o valor ponderado da soma das entradas.

- Camada de saída com Softmax: Após a passagem pela camada densa, o modelo utiliza a função de ativação **Softmax** na camada final para transformar as saídas da rede em uma probabilidade, que varia entre 0 e 1. A função **Softmax** é dada por:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Onde $\hat{y}_i$ é a probabilidade da classe i e z_i é a ativação da classe i.

3. Decisão final: O modelo classifica a instância como *"True"* ou *"False"* com base no valor de $\hat{y}$:

- Se $\hat{y} > 0.5$, a correspondência é classificada como *"True"*. - Caso contrário, a correspondência é classificada como *"False"*.

6.1. Exemplo de execução

Para os dados fornecidos, o modelo gera uma probabilidade $\hat{y}$ após o cálculo das ativações nas camadas da rede. Vamos ilustrar o funcionamento da rede para essa instância:

1. O vetor de entrada $x = [0.8, 0.7, 0.9, 0.85, 0.5]$ é passado pela rede. 2. A primeira camada com a função ReLU processa o vetor de entrada, aplicando a transformação linear seguida de ReLU. 3. A camada final com Softmax calcula a probabilidade $\hat{y}$. 4. Com base no valor de $\hat{y}$, o modelo decide se a correspondência é *"True"* ou *"False"*.

6.1.1. Resultados da avaliação

Suponha que o modelo calcule uma probabilidade $\hat{y} = 0.75$ para essa instância. Como $\hat{y} > 0.5$, a correspondência é classificada como *"True"*.

Portanto, o resultado da avaliação para essa instância seria:

$$\boxed{\text{Classificação: True}}$$

Esse processo de classificação é aplicado a todas as instâncias de teste, permitindo que o modelo classifique automaticamente pares de ontologias como correspondentes ou não, com base nas probabilidades previstas pela rede neural.

6.1.2. Detalhamento das Etapas

Durante o treinamento e a avaliação, o cálculo das ativações nas camadas da rede é fundamental para ajustar os pesos da rede e minimizar a função de erro. O modelo usa o algoritmo de **gradiente descendente estocástico (SGD)** para ajustar esses parâmetros e melhorar sua precisão ao classificar pares de ontologias. O treinamento envolve a minimização da função de **log-loss**, que é calculada com base nas diferenças entre as probabilidades previstas ($\hat{y}$) e os rótulos reais (y).

$$\text{Loss} = -\left(y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})\right)$$

Esse processo é repetido até que o modelo converja para os melhores parâmetros (pesos e bias) que minimizam a loss, proporcionando uma melhor performance de classificação.

7. Hiperparâmetros do Modelo - Biblioteca Java Deeplearning4j

Diversos **hiperparâmetros** são usados para configurar e treinar o modelo de rede neural MLP (Multi-Layer Perceptron) com a biblioteca Deeplearning4j [Krohn et al. 2019] [Aggarwal et al. 2018]. Abaixo, estão detalhados os principais hiperparâmetros presentes no código.

- **Número de Features (`numInputs`)**
  - **Valor**: 6
  - **Descrição**: O número de entradas (ou características) que o modelo irá processar. Cada instância de treinamento possui 6 atributos ($Cosine, NGram, Lin, Max, MedMax, Dem$) para a classificação.
- **Número de Saídas (`numOutputs`)**
  - **Valor**: 2
  - **Descrição**: Como a tarefa é uma classificação binária (verdadeira ou falsa), o número de saídas corresponde a duas classes. Uma saída será para "False"e outra para "True".
- **Tamanho do Lote (`batchSize`)**
  - **Valor**: 64
  - **Descrição**: O número de amostras de dados que o modelo processa de uma vez durante o treinamento. O uso de mini-lotes (em vez de processamento de todas as amostras ao mesmo tempo) ajuda a melhorar a eficiência e a estabilidade do treinamento.
- **Número de epoch (`numEpochs`)**
  - **Valor**: 10
  - **Descrição**: O número de iterações completas sobre todo o conjunto de treinamento. O modelo treina por 10 epoch, o que significa que os dados são passados pelo modelo 10 vezes durante o treinamento.
- **Semente (`seed`)**
  - **Valor**: 123
  - **Descrição**: A semente é usada para inicializar os pesos de maneira controlada e garantir que os resultados sejam reprodutíveis. A semente fixa o estado inicial da rede neural para que, ao executar o código novamente, os resultados sejam consistentes.
- **Algoritmo de Otimização (`OptimizationAlgorithm`)**
  - **Valor**: `STOCHASTIC_GRADIENT_DESCENT`
  - **Descrição**: O algoritmo utilizado para otimização da rede neural. O SGD (Gradiente Descendente Estocástico) é um dos algoritmos mais comuns para ajustar os pesos da rede durante o treinamento. Ele ajusta os pesos após cada mini-lote de dados.
- **Atualizador (`Updater`)**
  - **Valor**: `Adam(0.006)`
  - **Descrição**: O otimizador Adam é uma versão aprimorada do SGD, que adapta as taxas de aprendizado para diferentes parâmetros durante o treinamento. O valor `0.006` representa a taxa de aprendizado

inicial, que controla a velocidade com que os pesos da rede são atualizados.

- **Função de Ativação (`Activation`)**
  - **Camada densa (primeira camada):**
    - **Valor**: `Activation.RELU`
    - **Descrição**: A função ReLU (Rectified Linear Unit) é uma função de ativação não linear comumente utilizada em redes neurais. Ela ajuda a introduzir não linearidades no modelo, permitindo que ele aprenda relações mais complexas nos dados.
  - **Camada de saída (última camada):**
    - **Valor**: `Activation.SOFTMAX`
    - **Descrição**: A função de ativação Softmax é usada na camada de saída para converter os valores de ativação da rede em probabilidades que somam 1. Ela é particularmente útil para problemas de classificação multiclasse ou binária, como neste caso.
- **Inicialização dos Pesos (`WeightInit`)**
  - **Valor**: `WeightInit.XAVIER`
  - **Descrição**: A inicialização Xavier é uma técnica que ajusta os pesos da rede de forma que a variância das ativações se mantenha balanceada entre as camadas. Isso pode ajudar a acelerar o treinamento e melhorar a performance da rede.
- **Função de loss (`LossFunction`)**
  - **Valor**: `LossFunctions.LossFunction.NEGATIVELOGLIKELIHOOD`
  - **Descrição**: A função de loss utilizada é a **Negative Log-Likelihood** (NLL), que é adequada para problemas de classificação binária e multiclasse. Ela penaliza as previsões que estão longe dos rótulos reais.
- **Threshold de Classificação (`threshold`)**
  - **Valor**: 0.3
  - **Descrição**: Esse threshold é usado durante a classificação para determinar a classe prevista. Se a probabilidade da classe "True"for maior ou igual a 0.3, a instância será classificada como "True"; caso contrário, será classificada como "False". Esse valor pode ser ajustado dependendo do tipo de problema e dos dados.
- **Número de Neurônios na Primeira Camada (`nOut`)**
  - **Valor**: 50
  - **Descrição**: A camada densa inicial possui 50 neurônios. Esse número é um hiperparâmetro que pode afetar a capacidade da rede de aprender representações dos dados. Neurônios em camadas intermediárias geralmente ajudam a capturar padrões mais complexos.

Esses hiperparâmetros podem ser ajustados para melhorar a performance do modelo em diferentes cenários, dependendo do comportamento dos dados e da tarefa de classificação.

8. Implementação

O Programa *View Interactive Ontology Matching* foi implementado utilizando as seguintes APIs em Java:

Hiperparâmetro	Valor	Descrição
Número de Features (numInputs)	6	Número de características de entrada do modelo.
Número de Saídas (numOutputs)	2	Número de classes na tarefa de classificação binária.
Tamanho do Lote (batchSize)	64	Número de exemplos usados para atualizar os pesos por vez.
Número de epoch (numEpochs)	10	Número de vezes que o modelo será treinado sobre todos os dados.
Semente (seed)	123	Garante reprodutibilidade nos resultados.
Algoritmo de Otimização (OptimizationAlgorithm)	STOCHASTIC_GRADIENT_DESCENT	Utiliza Gradiente Descendente Estocástico para otimização.
Atualizador (Updater)	Adam(0.006)	Usado para ajustar as taxas de aprendizado durante o treinamento.
Função de Ativação (Activation)	ReLU / Softmax	ReLU na camada densa e Softmax na camada de saída.
Inicialização de Pesos (WeightInit)	Xavier	Técnica de inicialização para melhorar o treinamento.
Função de loss (LossFunction)	NEGATIVELOGLIKELIHOOD	Função usada para calcular a loss durante o treinamento.
threshold de Classificação (threshold)	0.3	Limite para classificar como "True"ou "False".
Neurônios na Primeira Camada (nOut)	50	Número de neurônios na camada densa inicial.

Tabela 2. Resumo dos Hiperparâmetros do Modelo

- Eclipse Deeplearning4J (DL4J): um conjunto de projetos destinado a atender todas as necessidades de um aplicativo de deep learning baseado em JVM[4];
- Simmetrics: um conjunto de métricas de similaridade baseadas em strings [Chapman 2007];
- WS4J: um conjunto de métricas linguísticas baseadas na WordNet [Priyadi et al. 2019]; e
- Alignment: um conjunto de rotinas para manipulação de ontologias escritas em OWL [David et al. 2011].

9. Alinhamento gerado pelo Programa *View Interactive Ontology Macthing* - ontologia de origem *cmt* e ontologia de destino *Conference* em RDF/XML

```
<?xml version='1.0' encoding='utf-8' standalone='no'?>
<rdf:RDF xmlns='http://knowledgeweb.semanticweb.org/heterogeneity/alignment#'
         xmlns:rdf='http://www.w3.org/1999/02/22-rdf-syntax-ns#'
         xmlns:xsd='http://www.w3.org/2001/XMLSchema#'>
<Alignment>
  <xml>yes</xml>
  <level>0</level>
  <type>?*</type>
  <onto1>
    <Ontology rdf:about="cmt">
      <location>C:/Ontologias/cmt</location>
    </Ontology>
  </onto1>
<onto2>
    <Ontology rdf:about="conference">
      <location>C:/Ontologias/conference</location>
    </Ontology>
  </onto2>
   <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#Author'/>
      <entity2 rdf:resource='http://conference#Regular_author'/>
```

[4]https://github.com/deeplearning4j/deeplearning4j-examples

```
      <relation>=</relation>
      <measure rdf:datatype='http://www.w3.org/2001/
         XMLSchema#float'>1.0</measure>

    </Cell>
  </map>
  <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#Chairman'/>
      <entity2 rdf:resource='http://conference#Chair'/>
      <relation>=</relation>
      <measure rdf:datatype='http://www.w3.org/2001/
         XMLSchema#float'>1.0</measure>
    </Cell>
  </map>
  <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#Co-author'/>
      <entity2 rdf:resource='http://conference#
         Contribution_co-author'/>
      <relation>=</relation>
      <measure rdf:datatype='http://www.w3.org/2001/
         XMLSchema#float'>1.0</measure>

        </Cell>
  </map>


  <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#Conference'/>
      <entity2 rdf:resource='http://conference#Conference'/>
      <relation>=</relation>
      <measure rdf:datatype='http://www.w3.org/2001/
         XMLSchema#float'>1.0</measure>
    </Cell>
  </map>
  <map>
    <Cell>
       <entity1 rdf:resource='http://cmt#Document'/>
       <entity2 rdf:resource='http://conference#
          Conference_document'/>
      <relation>=</relation>
         <measure rdf:datatype='http://www.w3.org/2001/
            XMLSchema#float'>1.0</measure>
    </Cell>
  </map>
```

```
  <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#PaperAbstract'/>
      <entity2 rdf:resource='http://conference#Abstract'/>
          <relation>=</relation>

 <measure df:datatype='http://www.w3.org/2001/XMLSchem␣␣␣a#
    float'>1.0</measure>
    </Cell>
  </map>
  <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#Person'/>
      <entity2 rdf:resource='http://conference#Person'/>
      <relation>=</relation>
      <measure rdf:datatype='http://www.w3.org/2001/
         XMLSchema#float'>1.0</measure>
    </Cell>
  </map>
  <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#Preference'/>
      <entity2 rdf:resource='http://conference#
         Review_preference'/>
        <relation>=</relation>
        <measure                rdf:datatype='http://www.w3.
           org/2001/XMLSchema#float'>1.0</m
        easure>
    </Cell>
  </map>
  <map>
    <Cell>
      <entity1 rdf:resource='http://cmt#ProgramCommittee'/>
      <entity2 rdf:resource='http://conference#
         Program_committee'/>
      <relation>=</relation>
      <measure rdf:datatype='http://www.w3.org/2001/
         XMLSchema#float'>1.0</measure>
    </Cell>
  </map>
  <map>
    <Cell>
       <entity1 rdf:resource='http://cmt#Review'/>
      <entity2 rdf:resource='http://conference#Review'/>
      <relation>=</relation>
      <measure rdf:datatype='http://www.w3.org/2001/
         XMLSchema#float'>1.0</measure>
```

```
1
2     </Cell>
3   </map>
4   <map>
5     <Cell>
6       <entity1 rdf:resource='http://cmt#email'/>
7       <entity2
8
9       rdf:resource='http://conference#has_an_email'/>
10
11      <relation>=</relation>
12      <measure rdf:datatype='http://www.w3.org/2001/
           XMLSchema#float'>1.0</measure>
13    </Cell>
14  </map>
15 </Alignment>
16 </rdf:RDF>
```

10. Resultado da avaliação do alinhamento por meio do alinhamento de referência

Correspondência	Precisão	Cobertura	Medida F
VIEW-cmt-conference	1.000	0.733	0.846
VIEW-cmt-confOf	1.000	0.437	0.609
VIEW-cmt-Edas	1.000	0.769	0.870
VIEW-cmt-Ekaw	1.000	0.545	0.706
VIEW-cmt-Iasted	1.000	1.000	1.000
VIEW-cmt-Sigkdd	1.000	0.833	0.909
VIEW-conference-confOf	1.000	0.800	0.889
VIEW-conference-Edas	1.000	0.765	0.867
VIEW-conference-Ekaw	1.000	0.760	0.864
VIEW-conference-Iasted	1.000	0.429	0.600
VIEW-conference-Sigkdd	1.000	0.800	0.889
VIEW-confOf-Edas	1.000	0.632	0.774
VIEW-confOf-Ekaw	1.000	0.850	0.919
VIEW-confOf-Iasted	1.000	0.667	0.800
VIEW-confOf-Sigkdd	1.000	0.714	0.833
VIEW-Edas-Ekaw	1.000	0.652	0.789
VIEW-Edas-Iasted	1.000	0.579	0.733
VIEW-Edas-Sigkdd	1.000	0.533	0.696
VIEW-Ekaw-Iasted	1.000	0.700	0.824
VIEW-Ekaw-Sigkdd	1.000	0.818	0.900
VIEW-Iasted-Sigkdd	1.000	1.000	1.000
VIEW - Conference Dataset	1.000	0.800	0.889

Tabela 3. Tabela de Correspondências Geradas pelo Programa *View Interactive Ontology Macthing*: Avaliação em Precisão, Cobertura e Medida F

11. Considerações Finais

Para resolver os problemas de heterogeneidade na integração de ontologias no domínio da segurança de plantas de usinas nucleares (como Angra 1 e Angra 2), foi proposta a utilização da técnica de clusterização com o algoritmo K-Means, juntamente com um tipo de rede neural artificial em Deep Learning: *MLP* (Perceptron Multicamadas). Para implementar essa abordagem, foi desenvolvido o Programa *View Interactive Ontology Matching*, que emprega essas técnicas no processo de alinhamento de ontologias.

A importância da simulação da avaliação realizada por um especialista foi destacada, sendo este processo conduzido pela uma rede neural artificial. Os resultados obtidos mostraram-se promissores, com valores elevados nas métricas de precisão, cobertura e medida-F.

Os resultados experimentais demonstram que o *View* é eficaz no alinhamento de ontologias heterogêneas, conseguindo determinar alinhamentos de alta qualidade.

No futuro, outras abordagens poderão ser exploradas em conjunto com a técnica utilizada, com o objetivo de aprimorar as métricas de Cobertura e Medida-F. Além disso, há planos para avaliar o *View* utilizando pares de ontologias do domínio de segurança cibernética.

Referências

Aggarwal, C. C. et al. (2018). *Neural networks and deep learning*, volume 10. Springer.

Alguliyev, R., Imamverdiyev, Y., e Sukhostat, L. (2018). Cyber-physical systems and their security issues. *Computers in Industry*, 100:212–223.

Chapman, S. (2007). String similarity metrics for information integration. *Sam's String Metrics*.

Cunha, L. (2022). Deep learning with python (2ª ed) - françois chollet - manning, outubro 2021, 504 pp. *Interações: Sociedade e as novas modernidades*, pp. 113–115.

David, J., Euzenat, J., Scharffe, F., e Trojahn dos Santos, C. (2011). The alignment api 4.0. *Semantic web*, 2(1):3–10.

de Sá, A. O., Carmo, L. F. R. d. C., e Machado, R. C. S. (2017). Covert attacks in cyber-physical control systems. *IEEE Transactions on Industrial Informatics*, 13(4):1641–1651.

Ghazani, S. H. H. N., Lotf, J. J., e Aliguliyev, R. (2012). A study on qos models for mobile ad-hoc networks. *International Journal of Modeling and Optimization*, 2(5):634.

Goodfellow, I., Bengio, Y., e Courville, A. (2016). *Deep Learning*. MIT Press. http://www.deeplearningbook.org.

ISO (2013). *ISO/IEC 27002:2013 - Information technology — Security techniques — Code of practice for information security controls*. International Organization for Standardization.

Krohn, J., Beyleveld, G., e Bassens, A. (2019). *Deep learning illustrated: a visual, interactive guide to artificial intelligence*. Addison-Wesley Professional.

Lin, W. e Haga, R. (2021). Matching cyber security ontologies through genetic algorithm-based ontology alignment technique. *Security and Communication Networks*, 2021.

Mozzaquatro, B. A., Agostinho, C., Goncalves, D., Martins, J., e Jardim-Goncalves, R. (2018). An ontology-based cybersecurity framework for the internet of things. *Sensors*, 18(9).

Priyadi, Y., Djunaidy, A., e Siahaan, D. (2019). Requirements dependency graph modeling on software requirements specification using text analysis. In *2019 1st International Conference on Cybernetics and Intelligent System (ICORIS)*, volume 1, pp. 221–226.

Qamar, R. e Zardari, B. A. (2023). Artificial neural networks: An overview. *Mesopotamian Journal of Computer Science*, 2023:124–133.

Santos, F. e Mello, C. E. (2022). Matching network of ontologies: A random walk and frequent itemsets approach. *IEEE Access*, 10:44638–44659.

Tyagi, M. (2024). Exploring the potency of neural networks: A thorough examination of deep learning techniques. In *2024 First International Conference on Technological Innovations and Advance Computing (TIACOMP)*, pp. 327–332.

Xue, X., Tan, W., e Tian, Y. (2022). Matching cybersecurity ontologies on internet of everything through coevolutionary multiobjective evolutionary algorithm. *Sec. and Commun. Netw.*, 2022.

Zamazal, O. e Svátek, V. (2017). The ten-year ontofarm and its fertilization within the onto-sphere. *Journal of Web Semantics*, 43:46–53.

Zeadally, S. e Jabeur, N. (2016). *Cyber-physical system design with sensor networking technologies*. Institution of Engineering and Technology.

Zeng, J., Yang, L. T., Lin, M., Ning, H., e Ma, J. (2020). A survey: Cyber-physical-social systems and their system-level design methodology. *Future Generation Computer Systems*, 105:1028–1042.

Zengeya, T. e Vincent Fonou-Dombeu, J. (2024). A review of state of the art deep learning models for ontology construction. *IEEE Access*, 12:82354–82383.